

Adding a Digital Output — Extending Your File Further

This guide assumes you've been through [1 DigitalInput.md](#) and [2 VoltageRatio.md](#) .

We're extending that file once more, to handle a *third* Phidget channel alongside the other two: a **digital output** (to power an LED wired to a Phidget hub), controlled by clicking a button in the page.

This is the first channel in the workshop that *writes* to hardware instead of only reading from it — data will flow from a click in the browser out to the physical device, not just the other way around.

Step 1 — Add CSS for the output button

Your `<style>` block now has a `/* Button visual */` section and a `/* Voltage ratio visual */` section.

Add a new `/* Output control button */` section after them:

```
/* Output control button */
#outputButton {
  width: 180px;
  height: 180px;
  border-radius: 50%;
  margin: 32px auto;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.2rem;
  font-weight: bold;
  color: white;
  background: #8a8f98;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
  transition: background-color 0.1s ease, transform 0.1s ease;
  user-select: none;
  border: none;
  cursor: pointer;
}

#outputButton:disabled {
  cursor: not-allowed;
  opacity: 0.6;
}

#outputButton.on {
  background: #3a9d4f;
}

#outputButton.off {
  background: #8a8f98;
}
```

This is the same colored-circle look as `#buttonVisual` , but this time the circle itself is a real `<button>` element (see Step 2), not a plain `<div>` used only for display. That's why there's more here than `#buttonVisual` had:

- `border: none; cursor: pointer;` strip the default browser button chrome and show a hand cursor, so it reads as clickable.
- `:disabled` dims it and swaps to a "not allowed" cursor. We disable this button until we're connected — clicking it before there's a device to control wouldn't do anything useful, and now the button's own appearance says so.
- `.on` / `.off` are the same idea as `.pressed` / `.neutral` — two mutually exclusive classes toggled from JavaScript to show which state we're in.

Step 2 — Add markup for the output button

You now have a `Button State` section and a `Voltage Ratio` section in the body. Add a new section after them:

```
<!-- The digital output visual-->
<section>
  <h2>Digital Output</h2>
  <button id="outputButton" class="off" type="button" disabled>OFF</button>
</section>
```

A few things to notice here that differ from the elements added in `2_VoltageRatio.md` :

- **It's a `<button>` , not a `<div>`** . Every visual element added so far has been for *display only* — JavaScript wrote to them, but the user never interacted with them directly. This one is the opposite: it's the thing the user clicks to change the hardware's state.
- **`type="button"`** . This element lives in the body alongside `<form id="connectionForm">` but is not part of that form. Without `type="button"` , a `<button>` would need to be outside any `<form>` to avoid accidentally triggering a submit — being explicit here avoids any ambiguity if the markup ever gets rearranged.
- **`disabled` by default**. It starts unusable, matching `disconnectButton` in the connection form — both only make sense once we're connected.

Step 3 — Add an element reference

In the `// Elements` block, you already have references for the button visual and the ratio visual elements. Add one more line after them, for the button from Step 2:

```
const outputButton = document.getElementById("outputButton");
```

Step 4 — Add a third Phidget object variable

You already have:

```
let digitalInput;
let voltageRatioInput;
```

Add a third declaration after them:

```
let digitalOutput;
```

Step 5 — Add a line to `setConnectedUI`

Find `setConnectedUI` :

```
function setConnectedUI(isConnected) {
  statusText.textContent = isConnected ? "Connected" : "Disconnected";
  statusText.className = isConnected ? "connected" : "disconnected";
  connectButton.disabled = isConnected;
  disconnectButton.disabled = !isConnected;
  serverAddressInput.disabled = isConnected;
  serverPortInput.disabled = isConnected;
}
```

Add one new line at the end, leaving everything else exactly as it is:

```
function setConnectedUI(isConnected) {
  statusText.textContent = isConnected ? "Connected" : "Disconnected";
  statusText.className = isConnected ? "connected" : "disconnected";
  connectButton.disabled = isConnected;
  disconnectButton.disabled = !isConnected;
  serverAddressInput.disabled = isConnected;
  serverPortInput.disabled = isConnected;
  outputButton.disabled = !isConnected;
}
```

This is what makes the `disabled` attribute from Step 2 actually turn on and off — `outputButton.disabled = !isConnected` means "disabled while *not* connected," the same rule already used for `disconnectButton` .

Step 6 — Add a display helper for the output button

You already have `setButtonVisual` and `setRatioVisual` . Add a new function after them:

```
function setOutputButtonVisual(isOn) {
  outputButton.classList.toggle("on", isOn);
  outputButton.classList.toggle("off", !isOn);
  outputButton.textContent = isOn ? "ON" : "OFF";
}
```

Our output only ever has two states, `true` or `false` , so this function is really the same shape as `setButtonVisual` , with `PRESSED` / `NEUTRAL` renamed to `ON` / `OFF` and `pressed` / `neutral` renamed to `on` / `off` . Same shape, different vocabulary — because the thing it's describing (an on/off circuit) is the same shape as the thing `setButtonVisual` describes (an on/off button), even though one is an input and the other is an output.

Step 7 — Add the digital output channel inside

`connectToPhidgetServer`

Inside `connectToPhidgetServer`, you already open the digital input and voltage ratio channels, ending with something like:

```
async function connectToPhidgetServer(address, port) {  
  ...  
  
  await connection.connect();  
  
  // await openDigitalInput();  
  await openVoltageRatioInput();  
  
  setConnectedUI(true);  
}
```

Add a method to open the digital output channel, and call this method from `connectToPhidgetServer`. Again, comment out the call to `openVoltageRatioInput()`. Phidget can handle multiple open channels, but we only have one wire for the demo today.

```
async function connectToPhidgetServer(address, port) {  
  ...  
  
  await connection.connect()  
  
  //await openDigitalInput();  
  //await openVoltageRatioInput();  
  await openDigitalOutput();  
  
  setConnectedUI(true);  
}  
  
async function openDigitalOutput() {  
  digitalOutput = new phidget22.DigitalOutput();  
  
  digitalOutput.isHubPortDevice = false;  
  digitalOutput.hubPort = 0;  
  digitalOutput.channel = 0;  
  
  digitalOutput.onAttach = async () => {  
    updateRawStatePanel({  
      status: "Attached",  
      deviceName: digitalOutput.deviceName,  
      serialNumber: digitalOutput.deviceSerialNumber,  
      channel: digitalOutput.channel,  
      state: digitalOutput.state,  
    });  
    setOutputButtonVisual(digitalOutput.state === true);  
  };  
  
  digitalOutput.onDetach = () => {  
    updateRawStatePanel({ status: "Device detached" });  
    setOutputButtonVisual(false);  
  };  
}
```

```

    }

    await digitalOutput.open(5000);
  }

```

The most important thing to notice here is what's **missing** compared to the other two channels: there's no third callback like `onStateChange` or `onVoltageRatioChange`. Those existed because those channels are *inputs* — something in the physical world changes them, and the library needs a way to tell us "hey, that just happened." A `DigitalOutput` doesn't work that way: nothing changes its state except *us*, by calling a method on it (in Step 9). There's no external event to listen for, so there's no change callback to write. `onAttach` and `onDetach` are still here because "did we successfully connect to the device" is still something that happens to us, regardless of direction — and `onAttach` reads `digitalOutput.state` to sync the visual, useful in case the output was already on from a previous session.

The other thing to notice is that this channel is addressed differently than `digitalInput` and `voltageRatioInput` above it. Those two set `isHubPortDevice = true` and `hubPort`. This one sets `hubPort` and `channel`, with no `isHubPortDevice` at all. Both `hubPort` and `channel` tell the Phidget library *which* physical channel to attach to, but `isHubPortDevice` changes what "physical channel" even means:

- `isHubPortDevice = true` means the hub's port itself is the device — there's no smart peripheral on the other end, just a bare wire (a switch, an LED) connected straight to the port's raw pins. In that case the hub port number *is* the whole address, which is why `digitalInput` and `voltageRatioInput` only ever need `hubPort`.
- **A real VINT peripheral — like your OUT1100 — is not a hub port device.** It's a separate board with its own onboard controller that plugs into a hub's VINT port and identifies itself to the software as an actual device, independent of what's wired directly to the port. So it isn't addressed by treating the port as the device; it's addressed the normal VINT way, with `isHubPortDevice` left unset (`false`).
- `hubPort` still says which physical port the device is plugged into — that part doesn't go away just because it's a VINT device.
- `channel` is needed on top of that because the OUT1100 exposes 4 outputs through that one connection. `channel = 0` means "the first output," `channel = 1` the second, and so on — `hubPort` picks the device, `channel` picks which of its outputs you mean.

Which of these you need depends on your hardware. A bare-wire sensor/actuator on a hub port needs `isHubPortDevice / hubPort`, the way `digitalInput` and `voltageRatioInput` are set up. A multi-channel VINT device like the OUT1100 needs `hubPort` (to find the device) plus `channel` (to pick the output), with `isHubPortDevice` left off — as the code above does.

Step 8 — Add cleanup for the digital output channel on disconnect

You already have something like:

```

async function disconnectFromPhidgetServer() {
  if (digitalInput) {
    try {
      await digitalInput.close();
    } catch (error) {
      // Channel may already be closed – nothing to do here.
    }
    digitalInput = null;
  }
}

```

```

if (voltageRatioInput) {
  try {
    await voltageRatioInput.close();
  } catch (error) {
    // Channel may already be closed – nothing to do here.
  }
  voltageRatioInput = null;
}
setButtonVisual(false);
setRatioVisual(null);
setConnectedUI(false);
updateRawStatePanel({ status: "Disconnected" });
}

```

Add a matching block for the digital output channel, and a call to reset its visual, without touching the existing lines:

```

async function disconnectFromPhidgetServer() {
  if (digitalInput) {
    try {
      await digitalInput.close();
    } catch (error) {
      // Channel may already be closed – nothing to do here.
    }
    digitalInput = null;
  }

  if (voltageRatioInput) {
    try {
      await voltageRatioInput.close();
    } catch (error) {
      // Channel may already be closed – nothing to do here.
    }
    voltageRatioInput = null;
  }

  if (digitalOutput) {
    try {
      await digitalOutput.close();
    } catch (error) {
      console.warn(error);
      // Channel may already be closed - nothing to do here.
    }
    digitalOutput = null;
  }
  setButtonVisual(false);
  setRatioVisual(null);
  setOutputButtonVisual(false);
  setConnectedUI(false);
  updateRawStatePanel({ status: "Disconnected" });
}

```

Each channel gets its own guarded close block, since any of them could already be closed or never have successfully opened. `setOutputButtonVisual(false)` resets the button to `OFF` on disconnect — and `setConnectedUI(false)` disables it again at the same time, so the page can't be left showing a clickable button with nothing behind it.

Step 9 — Add a click handler that actually controls the hardware

The submit handler and `disconnectButton` listener at the bottom of the script stay exactly as they are — they don't need any changes. But now we add something genuinely new at the very end: a listener that writes to a Phidget channel instead of only reading from one.

Add this after the existing `disconnectButton.addEventListener(...)` line:

```
outputButton.addEventListener("click", async () => {
  if (!digitalOutput) {
    return;
  }

  const nextState = !digitalOutput.state;

  try {
    await digitalOutput.setState(nextState);
    setOutputButtonVisual(nextState);
    updateRawStatePanel({
      status: "State change",
      deviceName: digitalOutput.deviceName,
      serialNumber: digitalOutput.deviceSerialNumber,
      channel: digitalOutput.channel,
      state: nextState,
      timestamp: new Date().toISOString()
    });
  }
  catch (error) {
    updateRawStatePanel({ status: "Error", setStateError: String(error) });
  }
});
```

Walking through it:

- **The guard clause.** `if (!digitalOutput) { return; }` protects against a click landing before `digitalOutput` has been created, or after it's been set back to `null` on disconnect. In practice `outputButton.disabled` (Step 5) already prevents clicks in those states, but a disabled button can occasionally still receive a click in edge cases (e.g. a click that started before the button was disabled), so the check inside the handler is the real safety net.
- `const nextState = !digitalOutput.state;` flips whatever the output's *current* state is. This reads the channel's own `.state` property — the same property `onAttach` read in Step 7 — rather than tracking a separate "is it on" variable ourselves. That matters: if the button's own tracked idea of the state ever drifted from the hardware's actual state, reading `.state` directly keeps them in sync instead of compounding an error.
- `await digitalOutput.setState(nextState);` is the new part conceptually: it's the first time in this series that we call a method that *changes* something on the physical device, rather than only reading a

property or registering a callback. Like `connection.connect()` and `digitalOutput.open()`, it's `async` because sending a command to hardware over the network takes time and can fail.

- **We only update the visual and raw state panel *after* `setState` succeeds** (not before it, and not optimistically). If the command fails — the device was unplugged mid-click, say — the `catch` block logs the error instead, and the button's visual stays at whatever it last confirmed, rather than showing "ON" for an output that's actually still off. This is a deliberate choice: it's slightly slower-feeling than updating the display immediately, but it means the page never lies about the hardware's real state.

Trying it yourself

1. Make sure a Phidget Network Server is running and reachable, with a digital output device all connected to their respective hub ports, connect an led to your digital output device to see changes made to the device.
2. Open your updated file in a browser.
3. Enter the server's address and port (default `localhost:8989`) and click Connect.
4. Watch the raw state panel as the output channel comes online. Click the output button — the physical output device should switch on and off along with the on-screen button. You should see the LED turn on and off as the state changes.

If the on-screen output button changes but the physical device doesn't, double-check the wiring and hub port — `digitalOutput.hubPort = 0` is only correct if your output device is actually plugged into port 0.

Bonus Round

A `DigitalOutput` only ever has two states, on or off. Try combining the Digital Output with the Voltage Ratio Input (slider) — for example, turning the output on only once the ratio crosses some threshold — to see how an input channel and an output channel can be wired together.